

Python i wielowątkowość

Interpreter Pythona rozwiązuje problem działania wielu wątków w ten sposób, że tylko jeden wątek może znajdować się w stanie wykonania w dowolnym momencie. Jest to realizowane za pomocą pojedynczej blokady: **Global Interpreter Lock (GIL)**, która jest murem (blokadą), pozwalającym tylko jednemu wątkowi na kontrolowanie interpretera Pythona w danym momencie. GIL nie jest widoczny dla programów jednowątkowych, ale może być problemem wydajności w kodzie wielowątkowym, ponieważ GIL pozwala na wykonanie tylko jednego wątku naraz, nawet w architekturze wielowątkowej z więcej niż jednym rdzeniem procesora.

```
import threading
def uf():
    print("-----")
    print(threading.current_thread().name)
    print(threading.get_ident())
    print(r'\_____/' )
    return
threads = []
for i in range(5):
    t = threading.Thread(target=uf)
    threads.append(t)
    t.start()
```

`t = threading.Thread(target=uf, args=(2, 3), daemon=True)`

```
import threading
def uf(liczba1, liczba2):
    print("Liczby: ",liczba1,liczba2)
    print("Wynik = ",int(liczba1) + int(liczba2))
    return
t = threading.Thread(target=uf,args=(2,3))
t.start()
```

```
import threading

def uf(liczba1, liczba2):
    print("Liczby: ",liczba1,liczba2)
    print("Wynik = ",liczba1 + liczba2)
    return

threads = []
for i in range(5):
    t = threading.Thread(target=uf,args=(2,3))
    threads.append(t)
    t.start() # nie ma .join(), non-daemon threads
```

Python i wielowątkowość

ZADANIE

Mamy program wykonywany na jednym wątku (lewa strona). Napisz program wykonywany na dwóch wątkach i sprawdź jaki będzie czas jego wykonywania.

```
import time
from threading import Thread
```

```
COUNT = 50000000
```

```
def countdown(n):
    while n>0:
        n -= 1
```

```
start = time.time()
countdown(COUNT)
end = time.time()
```

```
print('Time taken in seconds -', end - start)
```

```
import time
from threading import Thread
```

```
COUNT = 50000000
```

```
def countdown(n):
    while n>0:
        n -= 1
```

```
# utwórz dwa wątki t1 i t2, delegując im
# do wykonania połowę iteracji
```

```
start = time.time()
# tu wykonanie
end = time.time()
```

```
print('Time taken in seconds -', end - start)
```

Python i wielowątkowość

ZADANIE

Mamy program wykonywany na jednym wątku (lewa strona). Napisz program wykonywany na dwóch wątkach i sprawdź jaki będzie czas jego wykonywania.

```
import time
from threading import Thread
```

```
COUNT = 50000000
```

```
def countdown(n):
    while n>0:
        n -= 1
```

```
start = time.time()
countdown(COUNT)
end = time.time()
```

```
print('Time taken in seconds -', end - start)
```

```
import time
from threading import Thread
```

```
COUNT = 50000000
```

```
def countdown(n):
    while n>0:
        n -= 1
```

```
t1 = Thread(target=countdown, args=(COUNT//2,))
t2 = Thread(target=countdown, args=(COUNT//2,))
```

```
start = time.time()
t1.start()
t2.start()
t1.join()
t2.join()
end = time.time()
```

```
print('Time taken in seconds -', end - start)
```

Python i wielowątkowość

ZADANIE

Napisz program symulujący serwer z wątkiem-loggerem działającym w tle.

Wątek główny: wykonuje 3 "zadania serwera" (każde trwa 1 sekundę)

Wątek logger (daemon): co 0.3 sekundy wypisuje "Logger: ping... "

Część A:

Uruchom z `daemon=True` i BEZ `join()` — zaobserwuj, że logger zostaje zabity przed końcem.

Część B:

Dodaj `join()` na końcu i zobacz różnicę.

Część C:

Zmień na `daemon=False` i zobacz co się stanie bez `join()`.

```

1  import threading
2  import time
3
4  def logger():
5      """Wątek działający w tle - loguje 'ping' co 0.3 sekundy."""
6      for i in range(15): # Max 15 pingów (4.5 sekundy)
7          print(f"  Logger: ping {i+1}...")
8          time.sleep(0.3)
9      print("  Logger: KONIEC")
10
11
12 def zadanie_serwera(numer):
13     """Symuluje wykonanie zadania serwera."""
14     print(f"Serwer: Rozpocznam zadanie {numer}")
15     time.sleep(1)
16     print(f"Serwer: Zakończyłem zadanie {numer}")
17
18
19 if __name__ == "__main__":
20     print("=== START PROGRAMU ===")
21
22     # Utwórz wątek loggera z daemon=True
23     # t_logger = threading.Thread(...)
24
25     # Uruchom wątek loggera
26
27     # Wykonaj 3 zadania serwera (sekwencyjnie w głównym wątku)
28     for i in range(1, 4):
29         zadanie_serwera(i)
30
31     print("=== Główny wątek kończy pracę ===")
32
33     # (część B): Odkomentuj join() i zobacz różnicę
34     # t_logger.join()
35
36     print("=== KONIEC PROGRAMU ===")

```

Python i wielowątkowość

Najpopularniejszym sposobem ominięcia ograniczeń GIL jest użycie podejścia wieloprocessorowego, w którym zamiast wątków używa się wielu procesów. Każdy proces Pythona otrzymuje własny interpreter Pythona i przestrzeń pamięci. Python ma moduł wieloprocessorowy, który pozwala nam łatwo tworzyć procesy (jeden z przykładów w celu zbadania czasu wykonania):

```
from multiprocessing import Pool
import time

COUNT = 50000000

def countdown(n):
    while n>0:
        n -= 1

if __name__ == '__main__':
    pool = Pool(processes=2)
    start = time.time()
    r1 = pool.apply_async(countdown, [COUNT//2])
    r2 = pool.apply_async(countdown, [COUNT//2])
    pool.close()
    pool.join()
    end = time.time()
    print('Time taken in seconds -', end - start)
```